Requirements Gathering

E-Commerce Data Engine & AI Assistant with Integrated Data Quality Monitoring

Mohamed Ahmed Abdelghany · Adham Younes Abdelghany · Mahmoud Alaa Elbanna · Moaaz Mohamed Moustafa

Revised draft — 12 October 2026

Document control

Track: DEPI — Data Engineering

Team: Mohamed Ahmed Abdelghany · Adham Younes Abdelghany · Mahmoud Alaa Elbanna · Moaaz Mohamed Moustafa

Version: Revised draft — 12 October 2026

Status note: Thresholds and service choices marked as proposed must be confirmed after profiling the exact dataset files and agreeing the implementation design.

1. Purpose and stakeholders

This document defines functional and non-functional requirements for the Olist-based data platform, analytics and AI features, and data-quality/trust monitoring. Stakeholders are customers using the assistant, business managers using BI, and data engineers maintaining the pipeline.

2. Dataset and assumptions

The source is the Olist Brazilian E-Commerce public dataset. It contains historical Brazilian e-commerce records; monetary values are treated as BRL and reviews are primarily Portuguese. The source has no true inventory snapshot and no reliable human-readable product names/descriptions in the core product table; the assistant therefore uses product IDs and translated categories, not fabricated product names. Synthetic inventory is for demonstration only.

Shipping metrics are derived from available order timestamps, shipping_limit_date, and freight_value. Derived measures must be documented with formula and null-handling rules. Processing is scheduled batch, not real-time streaming.

Any counts recorded in the initial draft (including duplicate IDs, missing timestamps, zero-value payments, and reconciliation mismatches) are preliminary profiling observations. Recalculate them from the exact downloaded files before final submission.

3. Data profiling and known conditions

Orders without carrier handover timestamp: Expected for orders that were not shipped; flag delivered orders missing the timestamp.

Orders without customer delivery timestamp: Expected for non-delivered orders; flag delivered orders missing the timestamp.

Orders without order items: Keep for order-status analysis but exclude from sales and item-price metrics.

Orders without payment record: Flag for investigation; do not automatically infer payment amount.

Orders without review: Expected; not a pipeline failure.

Multiple reviews or duplicate review IDs: Do not blindly delete records based only on duplicate review_id. Define a deterministic deduplication key and policy. Preserve raw records; in curated data, choose the latest valid review per order only if the business rule is approved, and record discarded/retained counts.

Products without category: Map to a documented 'unknown' category and retain the original value.

Products missing weight/dimensions: Exclude only from analyses requiring those attributes; do not drop the product from all analysis.

Duplicate geolocation rows: Aggregate or deduplicate to one documented coordinate representation per zip prefix; retain a reproducible rule.

Zero-value payments / unknown payment type: Flag and report as baseline findings until investigated; do not automatically delete.

Payment reconciliation mismatch: Calculate using a defined formula and order eligibility rules. The 1.00 BRL tolerance is provisional and must be calibrated after recomputing clean-data baseline.

4. Functional requirements — data pipeline

FR-1.1 (High): Extract the agreed Orders, Order Items, Products, Customers, Sellers, Payments, Reviews, and shipping-related source tables/files.

FR-1.2 (High): Clean and standardise dates, types, text, missing values, and duplicates while preserving raw inputs and documenting every transformation.

FR-1.3 (High): Transform curated data into a documented dimensional model; define and test each fact table's grain to prevent double-counting.

FR-1.4 (High): Load the analytical warehouse in the selected Azure service. Maintain a local PostgreSQL development path until cloud access is confirmed.

FR-1.5 (Medium): If MongoDB is retained, store review documents there with stable order/review identifiers and a documented sync strategy; otherwise store reviews in the relational platform and update the architecture.

FR-1.6 (High): Schedule pipeline tasks through Airflow, with dependencies, retry policy, failure state, and clear logs.

FR-1.7 (Medium): Log run ID, start/end time, task status, source and output row counts, rejected/quarantined rows, and error messages.

FR-1.8 (High): Translate product category labels using the supplied translation file and explicitly map any remaining untranslated labels.

FR-1.9 (High): Apply an approved deterministic review deduplication rule. Preserve raw review rows and produce an audit count.

FR-1.10 (High): Derive shipping metrics from documented source fields; state how missing dates and negative/invalid intervals are handled.

FR-1.11 (High): Generate synthetic inventory with a fixed random seed, one record per product_id per defined snapshot, non-negative stock, and documented assumptions. Do not present synthetic stock as historical Olist truth.

FR-1.12 (Medium): Reduce geolocation data using a reproducible rule that produces one representative coordinate per zip prefix.

FR-1.13 (Low): Correct misspelled staging column names such as product_name_lenght and product_description_lenght while preserving a mapping to source names.

FR-1.14 (High): Keep monetary amounts in BRL, label them explicitly, and do not perform unrequested currency conversion.

5. Functional requirements — analytics and AI

FR-2.1 (High): Show top-selling products by units sold and item revenue as separate metrics; exclude cancelled/unavailable orders according to the approved sales-status rule.

FR-2.2 (High): Show average item price over time. Keep freight separate unless a chart explicitly reports total cost including freight.

FR-2.3 (High): Compute review sentiment using a Portuguese-capable model for text-bearing reviews and show score-based proxy sentiment separately for reviews without comment text.

FR-2.4 (Medium): Allow filtering by date range and product category; clearly state which date field drives each view.

FR-2.5 (High): Accept natural-language questions in the AI Shopping Assistant.

FR-2.6 (High): Ground answers in retrieved warehouse/review evidence and show product_id/category or source references where possible.

FR-2.7 (High): Support constrained questions such as products in a category under a budget; enforce numeric filters in SQL or deterministic code, not vector similarity alone.

FR-2.8 (Medium): Say that information is unavailable when no matching evidence exists; never invent product names, descriptions, live stock, or other unsupported attributes.

FR-2.9 (High): Support Portuguese review text and document evaluation on a manually checked sample. Report sample size and appropriate classification metrics.

FR-2.10 (High): Represent products by product_id and translated category because reliable product names/descriptions are not available in the core source table.

6. Functional requirements — DQ and Trust

FR-3.1 (High): Detect added, removed, renamed, or retyped columns against a versioned expected schema.

FR-3.2 (High): Detect numeric anomalies (e.g., price or quantity) using explainable statistical rules with appropriate grouping and baseline windows.

FR-3.3 (Low/Optional): Evaluate Isolation Forest only if time permits and a test demonstrates benefit beyond simpler rules.

FR-3.4 (High): Check cross-system consistency, including payment versus eligible order totals and synthetic inventory consistency with the documented stock model.

FR-3.5 (High): Detect unexpected currency labels or unit changes.

FR-3.6 (High): Assign each finding Warning or Critical severity with rule ID, affected dataset/column, observed/expected values, timestamp, and explanation.

FR-3.7 (High): Run required DQ checks before publishing data to trusted downstream tables. Critical findings block or quarantine the affected batch; warnings allow continuation only with a degraded status.

FR-3.8 (High): Show Healthy, Warning, or Critical status and a history of findings for data used by BI and the assistant.

FR-3.9 (Medium): Make critical alerts visible in the dashboard and run logs; use email/chat alerts only if configured and tested.

FR-3.10 (High): Implement payment reconciliation using a documented formula and provisional 1.00 BRL tolerance only after re-profiling; define eligible statuses and exclusions.

FR-3.11 (Medium): Separate known baseline findings from newly injected or newly observed anomalies.

7. Non-functional requirements and acceptance targets

NFR-1 Performance: Target full batch completion under 30 minutes on the documented development environment; record hardware and dataset version.

NFR-2 Assistant latency: Target typical query response under 10 seconds where model/provider limits permit; report median and p95 over a small test set.

NFR-3 Reliability: Retries and logs are required; failed tasks must not publish partial or corrupted curated data.

NFR-4 Data accuracy: Primary/foreign keys and mandatory-field checks pass according to the data dictionary; exceptions are explicitly documented rather than silently dropped.

NFR-5 DQ detection: All deliberately injected faults in the test suite are detected, with clean-baseline false positives measured and reported.

NFR-6 Scalability: Transformation interfaces should allow later migration to Spark if measured data volume requires it; Spark is not mandatory for the current dataset.

NFR-7 Portability: Docker Compose and setup documentation should start the local services with a small number of documented commands.

NFR-8 Maintainability: Modular code, version control, README, configuration examples, unit tests, and peer review.

NFR-9 Security: Secrets in environment variables or a local ignored .env file; commit only .env.example with placeholders.

NFR-10 Usability: Dashboard labels define metrics and filters; a non-technical user can interpret the main views.

NFR-11 Explainability: Every DQ finding states what failed, where, why, and what downstream action was taken.

8. Severity and batch policy

Healthy: All required critical checks pass and warning count is within the agreed policy; data can be published.

Warning: A non-critical check fails or a known baseline issue occurs. The batch may continue if the rule permits, but the affected consumer view must display a warning.

Critical: Schema-breaking issue, invalid units/currency, or material reconciliation failure. Block or quarantine affected outputs and set the Trust status to Critical until revalidation passes.

Each rule must explicitly specify severity and action; do not infer severity only from the count of failed checks. Keep raw input for diagnosis and make reruns idempotent.

9. User stories and acceptance criteria

US-1 — Customer budget recommendation: Given a category and maximum budget, the assistant returns only matching product IDs/categories from warehouse evidence; if none match, it says so.

US-2 — Review-based recommendation: Recommendations use stored review evidence and disclose whether sentiment comes from text or score proxy; if data status is Critical, the assistant warns or abstains.

US-3 — Manager sales view: Dashboard displays top products by units and revenue for the selected period, with the chosen order-status rule documented.

US-4 — Manager price trends: Dashboard displays average item price over time and identifies BRL and date basis.

US-5 — Manager sentiment: Dashboard displays positive/neutral/negative shares and distinguishes text-model sentiment from score proxy.

US-6 — Trust status: Trust dashboard gives per-consumer Healthy/Warning/Critical status with recent issues.

US-7 — Schema changes: Injected column rename/removal/retyping is detected, logged, and blocks publication when classified Critical.

US-8 — Numeric anomalies: Injected price/currency/inventory faults are detected with explanatory details.

US-9 — Cascading issues: A payment reconciliation failure is linked to the affected order/batch and shown to downstream consumers.

10. Traceability and test plan

ETL and warehouse: FR-1.1–FR-1.14; test row counts, keys, null handling, type casting, rerun idempotency, and fact-table grain.

BI dashboard: FR-2.1–FR-2.4; test metrics against direct SQL calculations for a fixed date/category selection.

RAG and sentiment: FR-2.3 and FR-2.5–FR-2.10; test Portuguese examples, no-text reviews, numeric budget constraints, no-match questions, unsupported questions, and critical trust status.

DQ/Trust: FR-3.1–FR-3.11; inject schema, price, currency, reconciliation, and inventory faults; confirm detection, severity, batch action, and status history.

Portability/security: NFR-7 and NFR-9; test fresh startup from instructions and verify secrets are not committed.

11. Constraints and out of scope

Project target dates in the team plan are code submission 30 Nov 2026 and final delivery 4 Dec 2026; confirm these against official DEPI guidance. True streaming, storefront/checkout, LLM pretraining, and automatic silent data correction are out of scope.

12. Decisions to close before design sign-off

Select the exact Azure warehouse service based on access and cost.

Confirm whether MongoDB is required for reviews or whether PostgreSQL alone is simpler.

Confirm Streamlit and whether Pandas-only is sufficient for the submitted scope.

Choose the Portuguese sentiment model and document the evaluation sample.

Choose the LLM, embedding model, vector store, and model fallback; separate SQL retrieval from semantic text retrieval.

Recompute source profiling counts and confirm reconciliation formula, tolerance, status exclusions, and baseline issues.

Approve synthetic inventory rules and make the synthetic nature visible in the dashboard and assistant.